

OPERATING SYSTEMS

Complete Notes — Easy Language

Process Management

Memory Management

CPU Scheduling

Deadlock

IPC

Virtual Memory

Disk Scheduling

Networking

Security

Linux vs Windows

Prepared for Exam Success | Read Once, Understand Everything

Based on Piyush Slashbyte OS Notes



Table of Contents

1. What is an Operating System?
2. Types of Operating Systems
3. Key Concepts — System Calls, Processes, Threads
4. Inter-Process Communication (IPC)
5. Concurrency, Synchronization & Deadlock
6. Process Management & Scheduling
7. CPU Scheduling Algorithms
8. Memory Management
9. Virtual Memory & Page Replacement
10. Virtual Machines, Storage, Networking
11. Disk Scheduling Algorithms
12. Backup, Security & Computing Environments

1 What is an Operating System?

An **Operating System (OS)** is software that acts as a **middleman between the user and the hardware**. Think of it as the manager of a company — it coordinates everything so that programs run smoothly without fighting over resources.

■ **EXAMPLE:** When you open Chrome, your OS allocates RAM, gives CPU time, manages the keyboard input you type, and shows the output on screen — all without you doing anything manually.

What does an OS do?

■ Process Management

Creates, runs, pauses and kills programs (processes).

■ Process Synchronization

Ensures multiple programs don't corrupt shared data.

■ Memory Management

Decides which program gets how much RAM.

■ CPU Scheduling

Decides which process uses the CPU and for how long.

■ File Management

Organizes files/folders on disk, controls who can read/write.

■ Security

Protects data from unauthorized access and malware.

Layers of a Computer System

Think of a computer as an onion with layers. The **user** is on the outside, the **hardware** (CPU, RAM) is at the core, and the OS sits just above hardware:

Layer	Who/What
Users	You — typing commands or clicking icons
Application Programs	Chrome, Word, Games
Utilities	File manager, antivirus
Operating System	■ The OS — coordinates everything below
Machine Language	Binary instructions (0s and 1s)

Layer	Who/What
Micro-Programming	Low-level hardware control
CPU / Memory / I/O	Actual physical hardware

2 Types of Operating Systems

Simple Batch OS

Runs jobs one after another, no user interaction during execution.

Real-world example: IBM 7090

■ **EXAMPLE:** Like a queue at a government office — next person only enters after the current one finishes.

Multi-programming Batch OS

Multiple programs loaded in memory; CPU switches to another when one waits for I/O.

Real-world example: IBM OS/360

■ **EXAMPLE:** Like a chef who chops veggies while water is boiling — never sitting idle.

Multi-tasking OS

CPU rapidly switches between tasks — so fast it feels like everything runs simultaneously.

Real-world example: Windows, macOS

■ **EXAMPLE:** You're listening to music, browsing the web AND downloading a file — all 'at once'.

Multi-processing OS

Uses multiple CPUs/cores. Tasks truly run in parallel.

Real-world example: UNIX, Linux

■ **EXAMPLE:** A restaurant with 4 chefs cooking 4 dishes at the exact same time.

Time Sharing OS

Each user/process gets a fixed time slot (quantum) on the CPU in rotation.

Real-world example: Cloud Servers

■ **EXAMPLE:** A teacher giving each student exactly 2 minutes to speak, going around the class.

Real-Time OS (RTOS)

Tasks MUST complete within a strict deadline. Used where timing is critical.

Real-world example: VxWorks, FreeRTOS

■ **EXAMPLE:** ABS brakes in a car — if the system doesn't respond in milliseconds, it fails.

Distributed OS

Many computers connected together work as one single system.

Real-world example: Apache Hadoop

■ **EXAMPLE:** Google's search uses thousands of servers that appear as one system to you.

Network OS

Manages shared resources (files, printers) across a network.

Real-world example: Windows Server

■ **EXAMPLE:** In an office, everyone accesses the same shared printer via the server.

UNIX & Linux — Special mention

UNIX and **Linux** are powerful, multi-user, multi-tasking OS commonly used in servers. Linux is essentially UNIX with open-source code — free for anyone to use and modify.

Feature	What it means	Example
Multi-user	Many people log in simultaneously	10 employees using the same server
Multi-tasking	Runs many processes at once	Web server handling 1000 requests
Time-sharing	Fair CPU time to all processes	No one process hogs the CPU
Networking	Built-in TCP/IP, SSH, FTP support	Hosting a website
Multi-processing	Uses all CPU cores	Faster video rendering

3 Key Concepts — Processes, Threads & IPC

System Calls

A **system call** is how a regular program asks the OS kernel to do something privileged (like reading a file or creating a process). Normal programs can't directly touch hardware — they must go through the OS for safety.

■ **EXAMPLE:** When VLC plays a video, it calls `open()` to open the file, `read()` to get data, and `write()` to send audio to your speaker — all via system calls.

Category	System Calls
File Operations	<code>open()</code> , <code>read()</code> , <code>write()</code> , <code>close()</code>
Process Control	<code>fork()</code> , <code>exec()</code> , <code>wait()</code> , <code>exit()</code>
Communication	<code>pipe()</code> , <code>socket()</code>
Memory	<code>malloc()</code> , <code>mmap()</code>

Processes

A **process** is a **program that is currently running**. The same program can have multiple processes — e.g., opening Chrome twice creates two separate Chrome processes, each with its own memory.

■ **EXAMPLE:** Program = recipe written on paper. Process = actually cooking that recipe right now. You can cook the same recipe multiple times simultaneously (multiple processes).

Process States

Every process goes through several states during its lifetime:

State	Meaning	Easy Analogy
New	Process just created, waiting to load into RAM	Job application submitted
Ready	Loaded in RAM, waiting for CPU turn	Sitting in interview waiting room
Running	CPU is executing this process	Currently being interviewed
Wait / Block	Waiting for I/O or event (not CPU)	Waiting for a document to be fetched
Suspend Ready	Moved to disk (RAM was full), but ready to run	Told to wait outside the building
Suspend Wait	Moved to disk AND still waiting for I/O	Left the building AND still waiting
Terminated	Process finished, all resources freed	Interview over, person left

Process Control Block (PCB)

Every process has a **PCB** — it's like the process's ID card stored by the OS. Also called **Process Descriptor**.

Attribute	What it stores	Why it matters
Process ID (PID)	Unique number for this process	OS uses it to find and manage the process
Program Counter	Address of next instruction to execute	Helps resume after interruption
Process State	New / Ready / Running / Waiting etc.	OS knows what to do with it
Priority	Number indicating urgency	Higher priority = gets CPU first
General Registers	Temporary data during execution	Saved when process is paused
List of Open Files	Files currently opened by this process	OS knows what to close on exit
List of Open Devices	Devices (printer, USB) in use	OS releases them when done

Threads

A **thread** is the smallest unit of execution. A single process can have **multiple threads** running simultaneously — they share the same memory but work independently.

■ **EXAMPLE:** A browser is one process. Rendering the webpage, running JavaScript, and downloading images are 3 separate threads inside that process — running at the same time!

- **User-level threads** — managed by a user library (faster, OS unaware)
- **Kernel-level threads** — managed directly by OS (more powerful, slower)

■ **KEY POINT:** Threads are faster than creating a new process because they share memory. Context switching between threads is cheaper than between processes.

4 Inter-Process Communication (IPC)

Processes are isolated — they can't directly access each other's memory. **IPC** provides mechanisms for processes to **talk to each other and coordinate**.

Pipe

Unidirectional data channel between two related processes.

■ **EXAMPLE:** Parent process sends data to child process — data flows only one way, like a water pipe.

Shared Memory

A block of RAM that multiple processes can read/write directly.

■ **EXAMPLE:** Two processes read/write to the same spreadsheet in memory — fastest IPC method.

Message Queue

Messages stored in a queue; receiver picks them up when ready.

■ **EXAMPLE:** WhatsApp messages — sender sends, messages wait in queue until receiver is online.

Direct Communication

Process directly names the other process to send/receive.

■ **EXAMPLE:** P1 explicitly says: 'send to P2'. One-to-one dedicated link.

Indirect Communication

Processes communicate via a shared 'mailbox' (port/channel).

■ **EXAMPLE:** Like an email inbox — multiple senders can drop messages, receiver picks up all.

Message Passing

Processes communicate using `send()` and `receive()` calls without shared memory.

■ **EXAMPLE:** Texting — you send a message, other person receives it. No shared address book needed.

FIFO (Named Pipe)

Full-duplex communication between unrelated processes via named pipe.

■ **EXAMPLE:** Two unrelated apps communicating via a named channel — bidirectional traffic.

Synchronization in IPC

When processes share resources, we need synchronization to avoid chaos. Here are the key mechanisms:

Mechanism	What it does	Real-world analogy
Mutual Exclusion	Only ONE process can access the critical section at a time	Only one person in a changing room at a time
Semaphore	Counter-based lock. P/wait() decreases counter; V/signal() increases it	A parking lot with 5 spots — counter tracks available spots
Barrier	All threads must reach this point before any can continue	Group of friends waiting at a meeting point before entering a theme park together
Spinlock	Process keeps checking in a loop if resource is free (busy waiting)	Repeatedly refreshing a ticket booking page until a seat becomes available

Mutex vs Semaphore

Feature	Mutex	Semaphore
What it is	A lock with ownership	A counter-based signal
Who can release	Only the thread that locked it	Any thread can signal
Count	Only 2 states: locked / unlocked	Any integer value (binary or counting)
Use case	Protecting a single shared resource	Managing a pool of N resources
Example	Database write access (1 writer at a time)	Printer pool (3 printers available)

Binary vs Counting Semaphore

Feature	Binary Semaphore	Counting Semaphore
Value range	0 or 1 only	Any non-negative integer
Purpose	Mutual exclusion (like a mutex)	Managing multiple identical resources
Resources managed	Only 1	N resources
Example	Access to a shared variable	3 printers available for 10 users

5 Concurrency, Synchronization & Deadlock

Concurrency vs Synchronization

Concurrency = multiple tasks making progress at the same time (not necessarily truly parallel). **Synchronization** = coordinating concurrent tasks so they don't corrupt shared data.

■ **NOTE:** Concurrency problem — Two people edit the same Google Doc at the same time. Without sync (locking), both could overwrite each other's changes!

Deadlock — The Four Horsemen

A **deadlock** occurs when a group of processes are all waiting for each other forever and none can proceed.

■ **NOTE:** Classic example: P1 holds Resource R1 and wants R2. P2 holds R2 and wants R1. Both wait forever — neither will release what they have. This is deadlock!

Deadlock requires ALL FOUR conditions to be true simultaneously:

- **1. Mutual Exclusion:** Resource can only be used by ONE process at a time. (e.g., a printer)
- **2. Hold and Wait:** A process holds one resource while waiting for another. (P1 holds R1, wants R2)
- **3. No Preemption:** Resources cannot be forcibly taken away — only voluntarily released.
- **4. Circular Wait:** A circular chain: P1 waits for P2, P2 waits for P3, P3 waits for P1.

Deadlock Handling Strategies

Strategy	How it works	Pros	Cons
Ignorance (Ostrich Algorithm)	OS pretends deadlock won't happen. If it does, restart manually.	Simple, fast	Deadlock may occur — system hangs
Prevention	Design system to violate at least one of the 4 conditions (e.g., order all resources)	Deadlock impossible	Wastes resources, limits flexibility
Avoidance (Banker's Algorithm)	OS checks if granting a request keeps system in a 'safe state' before allocating	No deadlock at runtime	Slow — checks every allocation
Detection & Recovery	Allow deadlock, detect it (via resource allocation graph), then kill a process or preempt	Flexible	Causes downtime during recovery

Banker's Algorithm — Real-life Analogy

Imagine a bank with ■1000 total. Customers can ask for money but the bank only gives money if doing so keeps enough reserve to service all other pending requests.

■ **EXAMPLE:** Bank has ■1000. Customer A wants ■500, B wants ■300, C wants ■200. Bank gives A ■500 (■500 left). Gives B ■300 (■200 left). Gives C ■200 (■0 left). At each step, bank confirms it still has enough to serve remaining customers. This ensures nobody is left stranded — same principle prevents OS deadlock!

Spooling — A Bonus Concept

Spooling (Simultaneous Peripheral Operations On-Line) temporarily stores data in a queue/buffer for a slower device like a printer. Processes dump their print jobs into the spool; the printer picks them up one by one.

■ **EXAMPLE:** When you print 5 documents quickly, they all go to the print queue (spool). The printer slowly prints them one by one while your computer is free to do other work.

6 Process Management & Scheduling

Types of Schedulers

Scheduler	Also called	What it does	When it runs
Long-term	Job Scheduler	Picks jobs from hard disk and loads them into RAM (ready queue)	Infrequently — when new job submitted
Short-term	CPU Scheduler	Picks which ready process gets the CPU next	Very frequently — every few milliseconds
Medium-term	Swapper	Moves processes between RAM and disk (swap space) to manage memory	When RAM is full or under-utilised

Process Queues

- **Job Queue:** All processes waiting on disk. Long-term scheduler picks from here and loads into RAM.
- **Ready Queue:** Processes in RAM, ready to run. Short-term scheduler picks the next one for CPU.
- **Waiting Queue:** Process is waiting for I/O. When I/O completes, it moves back to Ready Queue.

Important Time Metrics

These are used to measure how efficient a scheduling algorithm is:

Term	Definition	Formula
Arrival Time (AT)	When process enters the ready queue	Given
Burst Time (BT)	Total CPU time needed to complete the process	Given
Completion Time (CT)	When process finishes execution	Observed
Turnaround Time (TAT)	Total time from arrival to completion	$TAT = CT - AT$
Waiting Time (WT)	Time spent waiting in the ready queue	$WT = TAT - BT$
Response Time (RT)	Time from arrival to first CPU allocation	$RT = \text{first_start} - AT$
Throughput	Number of processes completed per unit time	$T = n / (\text{Max CT} - \text{Min AT})$

■ **EXAMPLE:** Quick example: Process arrives at AT=0, BT=5, CT=8. $TAT = 8 - 0 = 8$. $WT = 8 - 5 = 3$. The process waited 3 seconds for the CPU.

7 CPU Scheduling Algorithms

The **CPU scheduler** decides which process in the Ready Queue gets the CPU. Good scheduling means maximum CPU use, minimum waiting time.

FCFS — First Come First Served

Processes are served in the order they arrive. Simple queue.

■ **EXAMPLE:** Bank teller — first customer in line gets served first.

Advantages	Disadvantages
Simple, no starvation	Convoy Effect: A slow process holds up all fast ones behind it. High average waiting time.

SJF — Shortest Job First

Process with the SHORTEST burst time gets CPU first.

■ **EXAMPLE:** Supermarket express lane — person with fewer items goes first.

Advantages	Disadvantages
Minimum average waiting time	Starvation: Long processes may never run if short ones keep arriving. Hard to predict burst time.

Priority Scheduling

Each process has a priority number. Highest priority runs first.

■ **EXAMPLE:** Emergency room triage — critical patient treated before minor injury.

Advantages	Disadvantages
Important tasks run sooner	Starvation: Low-priority processes may never run. Solution: Aging (gradually increase priority over time).

Round Robin (RR)

Each process gets a fixed time quantum (e.g., 4ms). After that, CPU goes to next process.

■ **EXAMPLE:** Teacher asking each student to answer for exactly 2 minutes, cycling through the class.

Advantages	Disadvantages
Fair — every process gets equal CPU time	High turnaround time if quantum is too large; too much context switching if too small.

Multilevel Queue

Processes divided into different queues by type (e.g., system, interactive, batch). Each queue has its own algorithm.

■ **EXAMPLE:** Airport security: Business class lane, regular lane, priority lane — each managed separately.

Advantages	Disadvantages
Customized scheduling per process type	Processes stuck in their assigned queue. Inflexible.

Multilevel Feedback Queue

Like Multilevel Queue but processes CAN move between queues based on behaviour.

■ **EXAMPLE:** A student who studies hard gets promoted to the advanced class, and a slacker gets moved down.

Advantages	Disadvantages
Most flexible and adaptive	Most complex to implement.

8 Memory Management

Memory Management is how the OS allocates RAM to processes, keeps them from interfering with each other, and makes efficient use of available memory.

Types of Memory — Speed vs Cost

Type	Speed	Size	Cost	Example
Cache (L1/L2/L3)	Very Fast	KB to MB	Very Expensive	CPU internal cache
Main Memory (RAM)	Fast	GB	Moderate	DDR4 RAM sticks
Secondary Storage	Slow	TB	Cheap	HDD, SSD, USB drive

■ **EXAMPLE:** Cache is like your desk — fast but tiny. RAM is like a room — bigger but less instant. Hard disk is like a warehouse — huge but slow to access.

Partitioning

Memory is divided into partitions for processes. Two approaches:

- **Fixed Partitioning:** RAM divided into equal fixed-size blocks at boot time. Simple but causes Internal Fragmentation (wasted space inside a partition).
- **Dynamic Partitioning:** Partitions created as needed, sized to fit each process. Reduces internal fragmentation but causes External Fragmentation (scattered free gaps).

Fragmentation — The two types

Type	Cause	Example	Solution
Internal Fragmentation	Fixed-size allocation wastes unused space inside a partition	Process needs 3KB, given 4KB block — 1KB wasted inside	Use smaller partitions, compaction, memory pooling
External Fragmentation	Free memory scattered in small gaps that individually can't fit a process	3 free blocks of 1KB each but process needs 3KB contiguous — fails!	Compaction (merge free spaces), Paging, Segmentation

Paging

Paging divides both the process's logical memory and physical RAM into equal fixed-size chunks:

- **Page** = fixed chunk of virtual/logical memory (process side)
- **Frame** = same-sized chunk of physical RAM
- **Page Table** = map that says 'Page 3 of process is in Frame 7 of RAM'

■ **EXAMPLE:** Like a hotel: your room (frame) is assigned when you check in. Your booking (page) doesn't care which physical room you get — the front desk (page table) maps your booking to an actual room.

Memory Allocation Strategies

Strategy	How it works	Best for
First Fit	Allocate first hole big enough from start	Fast allocation
Best Fit	Allocate smallest hole that fits (least waste)	Minimum internal waste
Worst Fit	Allocate largest hole (leaves big leftover fragments)	When future large allocations expected

Segmentation

Segmentation divides a program into logical segments (code, data, stack, heap). Each segment can be different size and stored anywhere in memory.

■ **EXAMPLE:** Your code section is 2KB, data section is 5KB, stack is 1KB — each gets its own variable-size segment. Matches how programmers think about a program.

Aspect	Paging	Segmentation
Size of chunks	Fixed (equal pages/frames)	Variable (based on program logic)
Fragmentation	Internal (wasted inside page)	External (gaps between segments)
Basis	Physical memory convenience	Programmer's logical view
Transparency	Invisible to programmer	Visible — programmer defines segments

9 Virtual Memory & Page Replacement

Virtual Memory tricks each process into thinking it has a large, private chunk of RAM — even if actual RAM is small. The OS uses disk space as extra 'fake RAM'.

■ **EXAMPLE:** Your laptop has 8GB RAM but you run Chrome (4GB) + Photoshop (6GB) + VS Code (2GB). Virtual memory allows this by keeping only active pages in RAM and swapping the rest to disk.

- **Page Table:** Maps virtual addresses (what process sees) to physical RAM addresses.
- **Swapping:** OS moves pages from RAM to disk (swap file) when RAM is full, brings them back when needed.
- **Page Fault:** When a page needed isn't in RAM — OS must fetch it from disk (slow!).

Thrashing

Thrashing happens when the system is so low on RAM that it spends most of its time swapping pages in and out of disk instead of actually running programs — productivity collapses.

■ **NOTE:** Like a student who forgets everything overnight — every morning they re-read everything and have no time to actually study new material. Constant re-swapping with no real work!

Page Replacement Algorithms

When RAM is full and a new page is needed, the OS must choose which existing page to evict:

Algorithm	Rule	Analogy	Key Problem
FIFO	Remove the oldest page (first loaded)	First person in queue leaves first	Belady's Anomaly: More frames can cause MORE page faults!
Optimal (OPT)	Remove page that won't be used for the longest time	Remove the book you'll need last	Impossible in practice — future is unknown
LRU (Least Recently Used)	Remove the page not used for the longest time	Remove the book you haven't touched in months	Best practical algorithm — needs access history tracking
LFU (Least Frequently Used)	Remove the page accessed fewest times	Remove least popular item from menu	Old rarely-used pages may never get evicted
MRU (Most Recently Used)	Remove the most recently used page	Remove the book you just put back	Not ideal in most cases
Clock (Second-Chance)	Like FIFO but gives each page a second chance (reference bit)	Like FIFO queue but slow people get one more chance before being removed	Better than FIFO but not as good as LRU

10 Virtual Machines, Storage & Networking

Virtual Machines (VMs)

A **Virtual Machine** is a software-based computer that runs inside a physical computer. You can run Windows inside Linux, or 10 virtual servers on one physical server.

■ **EXAMPLE:** One physical server in a data centre runs 20 VMs — each serving a different client's website. If one VM crashes, the others are unaffected (isolation).

Hypervisor = software that creates and manages VMs:

- **Type 1 (Bare Metal):** Runs directly on hardware. Faster, used in data centres. (VMware ESXi, Hyper-V)
- **Type 2 (Hosted):** Runs on top of an existing OS. Slower but convenient. (VirtualBox, VMware Workstation)

Storage Solutions

Type	Description	Advantage	Disadvantage
HDD	Magnetic disks with spinning platters	Cheap, large capacity	Slow, fragile (moving parts)
SSD	Flash memory chips, no moving parts	Fast, durable, silent	Expensive per GB
NAS	Storage box connected to LAN	Shared access, easy backups	Slower than SAN
SAN	High-speed dedicated storage network	Very fast, enterprise-grade	Very expensive, complex
Cloud	Storage on internet servers	Accessible anywhere, scalable	Needs internet, security risks

HDD vs SSD — Quick Comparison

Feature	HDD	SSD
Technology	Spinning magnetic disks	Flash memory chips
Speed	Slow (especially random access)	Very fast
Noise	Audible spinning noise	Silent
Durability	Fragile — drops can damage	Shock-resistant
Cost per GB	Very cheap	More expensive
Power use	Higher	Lower
Best use	Bulk storage, backups	OS drive, fast applications

Networking Components

Device	OSI Layer	What it does	Uses
Hub	Layer 1 (Physical)	Broadcasts data to ALL connected devices	Old, inefficient — avoid
Bridge	Layer 2 (Data Link)	Connects two networks, filters by MAC	Reduces traffic between segments
Switch	Layer 2 (Data Link)	Sends data only to the correct device using MAC address	Modern LANs
Router	Layer 3 (Network)	Routes data between DIFFERENT networks using IP	Connecting LAN to internet
Firewall	Layer 3-7	Monitors & blocks unauthorized traffic	Network security
Access Point	Layer 1-2	Extends network wirelessly (Wi-Fi)	Wireless devices
Load Balancer	Layer 4-7	Distributes traffic across multiple servers	High-traffic websites

■ **EXAMPLE:** Hub = shout in a room (everyone hears). Switch = pass a note directly to one person. Router = post office that routes letters to different cities (different networks).

11 Disk Scheduling Algorithms

The **disk head** (read/write arm) must move to different tracks to read data. **Disk scheduling** optimizes how the head moves to minimize travel time and improve throughput.

Key metric: **Seek Time** = time for disk head to move to the correct track.

FCFS — First Come First Served

Requests are served in arrival order. No reordering.

■ **EXAMPLE:** Requests at tracks: 40, 10, 70, 20, 90. Head starts at 50. Moves: 50→40→10→70→20→90. Large zigzag movement!

Advantage	Disadvantage
Simple, fair, no starvation	Worst seek time — head zigzags all over the disk (convoy effect)

SSTF — Shortest Seek Time First

Always go to the closest track from current head position.

■ **EXAMPLE:** Head at 50. Requests at 40, 10, 70, 20, 90. Next: 40 (closest). Then 70, then 90, then 20, then 10.

Advantage	Disadvantage
Better seek time than FCFS	Starvation — requests far from current head position may never get served

SCAN (Elevator Algorithm)

Head moves in one direction, serves all requests, then reverses at the disk end.

■ **EXAMPLE:** Like an elevator: goes from floor 1 to 10 serving everyone, then comes back from 10 to 1.

Advantage	Disadvantage
No starvation, better than SSTF for even distribution	Requests just behind the head have to wait for a full round trip

LOOK

Like SCAN but head only goes as far as the last request, doesn't go to disk end.

■ **EXAMPLE:** Elevator that doesn't travel to the top/bottom floor if nobody is there.

Advantage	Disadvantage
More efficient than SCAN — avoids unnecessary travel	Slightly more complex than SCAN

C-SCAN (Circular SCAN)

Head moves in one direction only. After reaching the end, jumps back to the start without serving on the return.

■ **EXAMPLE:** Elevator that goes from ground to top, then teleports back to ground — never serves on the way down.

Advantage	Disadvantage
Uniform wait time for all requests	Jump-back wastes some time

C-LOOK

Like C-SCAN but jumps back to earliest request instead of disk beginning.

■ **EXAMPLE:** Elevator that jumps to the lowest waiting floor after reaching the top.

Advantage	Disadvantage
Most efficient — avoids unnecessary movement	Slightly complex to implement

12 Backup, Security & Computing Environments

Backup Types

Type	What's backed up	Speed	Recovery complexity	Example
Full Backup	Everything — all files & data	Slow	Easy (one file)	Sunday: copy entire server
Incremental	Only changes since LAST backup	Fast	Complex (need full + all incremental backups)	Mon: Mon's changes. Tue: Tue's changes only.
Differential	All changes since last FULL backup	Medium	Medium (need full + latest differential)	Mon: Mon's changes. Tue: Mon+Tue's changes.

■ **KEY POINT:** 3-2-1 Rule: Keep 3 copies of data, on 2 different media types, with 1 copy off-site. E.g., Local HDD + NAS + Cloud backup.

RTO and RPO

- **RTO (Recovery Time Objective):** Maximum acceptable time to restore service after failure. If RTO = 2 hours, you must be back up within 2 hours.
- **RPO (Recovery Point Objective):** Maximum acceptable data loss (in time). If RPO = 15 min, backups must be taken every 15 minutes.

■ **EXAMPLE:** Bank robbery at 3:00 PM. Bank's RTO = 30 minutes (back online by 3:30). RPO = 5 minutes (can lose max 5 minutes of transactions). Very strict!

Security Best Practices

Practice	What it does	Example
Role-Based Access Control (RBAC)	Users get only the permissions their role needs	Accountant can access financial data, not HR records
Multi-Factor Authentication (MFA)	Requires 2+ proofs of identity	Password + OTP on your phone
Data Encryption	Scrambles data so only authorized parties can read it	HTTPS encrypts your bank login; BitLocker encrypts hard drive
Incident Response Plan	Documented steps when security breach occurs	Malware detected → isolate machine → scan → restore from backup
Compliance Frameworks	Follow industry/legal data rules	HIPAA (healthcare), GDPR (EU privacy), PCI-DSS (payments)
Employee Training	Teach staff to recognize and avoid attacks	Phishing simulation emails to test & train employees

Windows vs Linux/Unix — Environment Comparison

Aspect	Windows	Linux / Unix
Architecture	GUI-first, user-friendly	CLI-first, powerful & flexible
File System	NTFS (journaling, permissions, large files)	ext4 (Linux), UFS (Unix)
CLI Tools	PowerShell, Command Prompt	Bash, Zsh — more powerful scripting
User Management	Active Directory (enterprise-friendly)	User/group model with fine-grained permissions
Cost / License	Paid, proprietary	Free and open-source (Ubuntu, CentOS)
Customization	Limited — GUI-driven	Extensive — everything configurable via CLI
Best for	Desktops, enterprise apps, gaming	Servers, cloud, developer machines, security

CLI Commands — Windows vs Linux

Operation	Windows	Linux/Unix
List files	dir	ls
Change directory	cd	cd
Create directory	mkdir	mkdir
Delete file	del	rm
Copy file	copy	cp
Move/Rename file	move	mv
Clear screen	cls	clear
List processes	tasklist	ps aux
Network config	ipconfig	ifconfig
Check connectivity	ping	ping
File permissions	icacls	chmod
File ownership	icacls	chown
Search text in file	find	grep
Check disk errors	chkdsk	fsck
Add user	net user	useradd



Quick Revision Cheat Sheet

OS	→	Interface between user and hardware. Manages CPU, memory, files, processes.
Process	→	A running program. Has PID, state (New→Ready→Run→Wait→Terminate), PCB.
Thread	→	Smallest execution unit within a process. Shares memory with other threads.
PCB	→	Process's ID card: stores PID, state, counter, registers, open files.
System Call	→	Way for user programs to request OS services. e.g. open(), fork(), write()
IPC Methods	→	Pipe, Shared Memory, Message Queue, FIFO, Direct/Indirect Comm, Message Passing
Deadlock 4 Conditions	→	Mutual Exclusion + Hold & Wait + No Preemption + Circular Wait
Banker's Algorithm	→	OS checks for 'safe state' before allocating resources. Avoids deadlock.
Semaphore ops	→	wait()/P() — decrement (acquire). signal()/V() — increment (release).
FCFS scheduling	→	First come first served. Simple but convoy effect.
SJF scheduling	→	Shortest job first. Min waiting time but starvation risk.
Round Robin	→	Time quantum for each process in rotation. Fair but high TAT.
TAT formula	→	$TAT = CT - AT$. $WT = TAT - BT$. $Response = first_start - AT$

Paging	→	Fixed-size pages (virtual) mapped to frames (physical RAM). No external fragmentation.
Virtual Memory	→	Uses disk as extra RAM. Page fault = page not in RAM, load from disk.
Thrashing	→	System spends more time swapping than executing. Caused by insufficient RAM.
LRU (page replace)	→	Replace page not used for longest time. Best practical algorithm.
SCAN disk algo	→	Elevator algorithm — head moves one direction to end, then reverses.
C-LOOK disk algo	→	Most efficient — only goes to last request then jumps back. No boundary waste.
3-2-1 Backup Rule	→	3 copies, 2 different media, 1 off-site.
RTO	→	Max time to restore service after failure.
RPO	→	Max acceptable data loss (time).
Type 1 Hypervisor	→	Runs directly on hardware. Faster. (VMware ESXi, Hyper-V)
Type 2 Hypervisor	→	Runs on OS. (VirtualBox, VMware Workstation)
Hub vs Switch	→	Hub = broadcasts to all. Switch = sends only to target device.
Router	→	Routes data between DIFFERENT networks. Uses IP addresses. Layer 3.
SSD vs HDD	→	SSD = fast, durable, expensive. HDD = slow, cheap, large capacity.